

6. Discussion

How does the actual speedup compare against the theoretical speedup?

- It follows Amdahl up to about 32 workers (93% at 8, 83% at 32) and drops to 58% at 64.
- Karp–Flatt shows where the divergence happens: e is flat while the measured curve tracks theory, and turns upward exactly where the curve stops tracking.
- So up to 32 ranks the loss is serial work plus the fixed gather and sort; beyond 32 it is communication that grows with rank count (§5.4).
- Gustafson sits below Amdahl because its fraction is taken from the parallel run; for this fixed- n experiment Amdahl is the appropriate model.

Will more MPI processes always increase the speedup?

- No. In theory Amdahl caps $n = 100M$ at 110.9× no matter how many processes there are.
- In practice the cluster flattens rather than collapses: speedup keeps rising all the way to 64 ranks, but 48 → 64 buys only 2% for 33% more hardware.
- Efficiency drops 87% → 77% → 65% → 37% at 8 / 16 / 32 / 64.
- Extending the Karp–Flatt trend, adding ranks past about 80 would make things slower.
- On a single machine with more ranks than cores, the curve falls outright (§3.6).

How does the workload distribution affect the speedup?

- By 23%, for free: **blockcyclic** reaches 12.35× against **block**'s 10.04× at 16 ranks.
- The reason is that testing a number costs more the bigger it is, so under **block** the rank holding the top slice finishes last and holds everyone up (imbalance 4.54×, which a simple $\sqrt{k}$ model predicted).
- A **dynamic** scheduler balances the load perfectly but is still slower, because imbalance was not the main problem and the scheduler spends a whole rank fixing it (§3.3).

Will the speed-up results be the same across different machines?

- No. The Amdahl ceiling carries over, because it depends only on the algorithm's phase mix and n . Nothing measured does.
- A CAAS core is about 4.7× slower than the laptop the code was developed on (53.7 s vs 11.4 s serial at 100M), yet the cluster reaches 23.9× where the laptop reached under 4×. Raw speed and scalability are separate things.
- What each hardware property controls:
 - Core count sets where the curve turns over.
 - Memory bandwidth sets how early efficiency starts to decay.
 - The network sets the 0.37 s gather floor; an InfiniBand interconnect would move the measurements closer to theory without changing a line of code.

Would you recommend Open MPI for prime searching, against POSIX threads or OpenMP?

- Yes, in hybrid form, when the problem needs more than one machine.
- Within a single node OpenMP is the better tool (13.39× vs MPI's 12.26× at 16 workers, with nothing to gather).

- But OpenMP cannot leave the node, and the fastest result in this study — 30.37x at 8x8 — is more than twice the best any shared-memory program can reach here.
- Within MPI the rule is: spend extra workers on threads, not ranks. Every 64-worker split with 8 or more threads per rank beats every split with 2 or fewer, by up to 35% (§4.3).
- One caveat: prime search is an unusually bad fit for message passing. It produces 46 MB of results for roughly 2 s of parallel compute, and all of it has to cross a process boundary. A workload that returns a single number would track Amdahl far more closely.